



## Classification: Back Propagation Neural Networks

**CS-4045: Deep Learning**

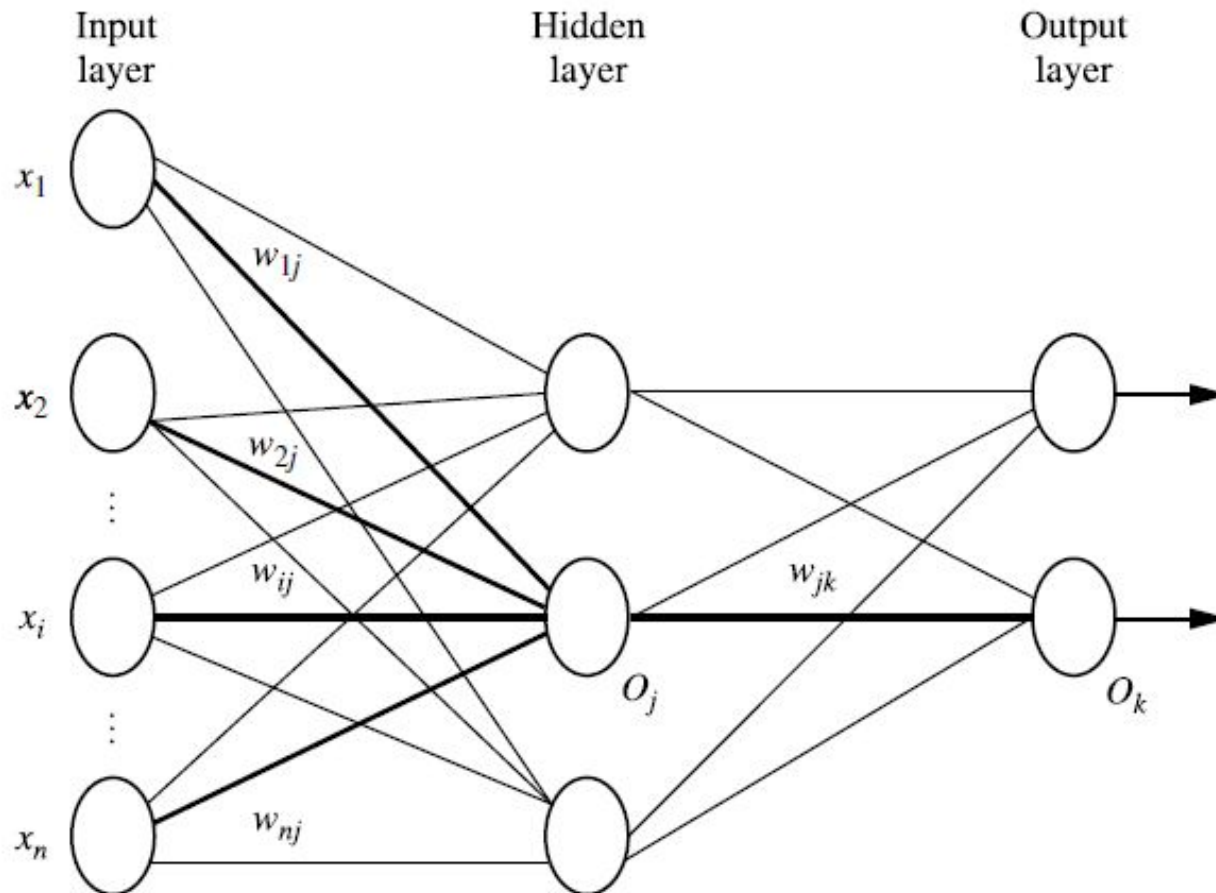
# Classification by Backpropagation

---

- Backpropagation: A **neural network** learning algorithm
- Started by psychologists and neurobiologists to develop and test computational analogues of neurons
- A neural network: A set of connected input/output units where each connection has a **weight** associated with it
- During the learning phase, the **network learns by adjusting the weights** so as to be able to predict the correct class label of the input tuples
- Also referred to as **connectionist learning** due to the connections between units

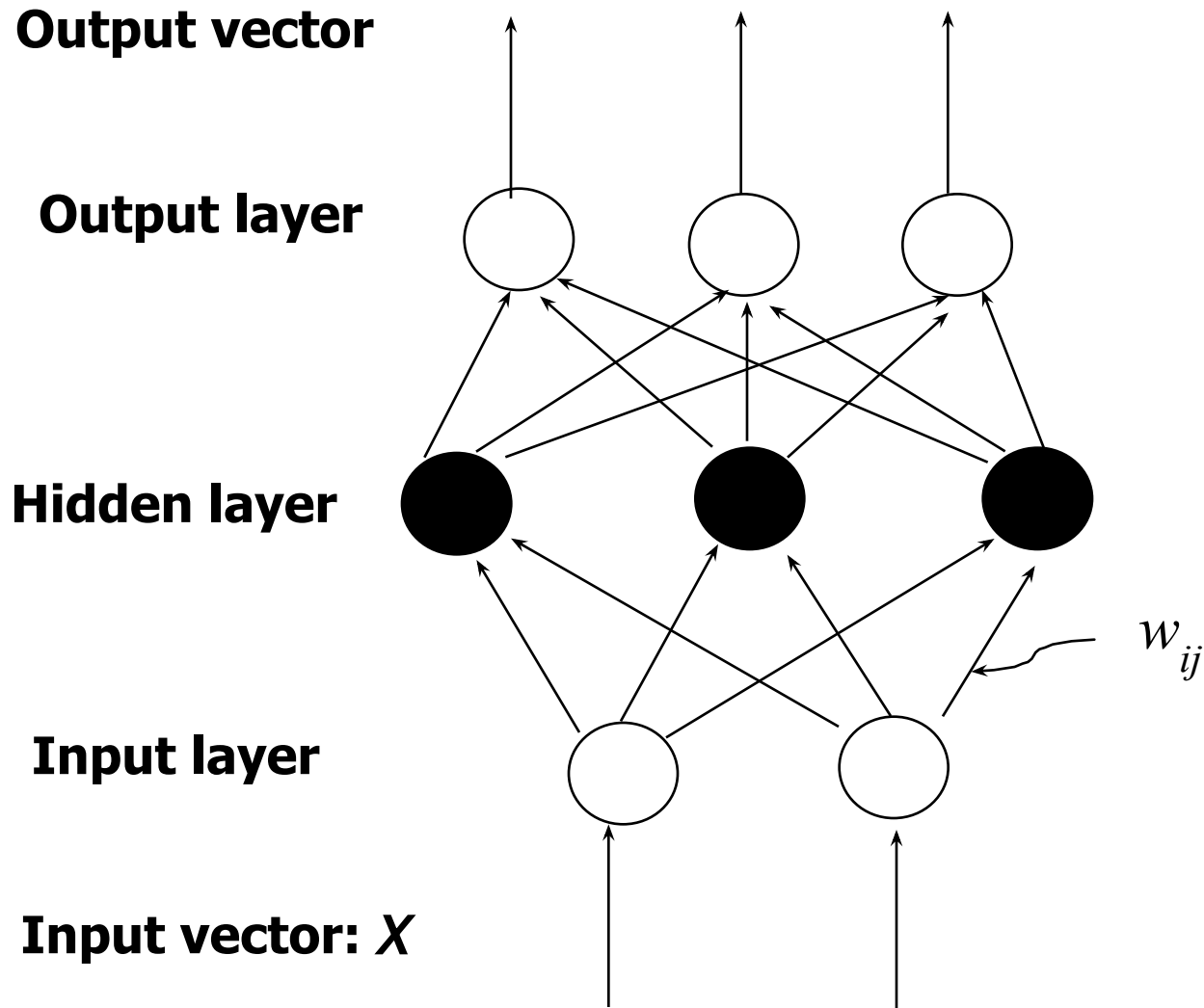
# Classification by Backpropagation

---



# A Multi-Layer Feed-Forward Neural Network

---



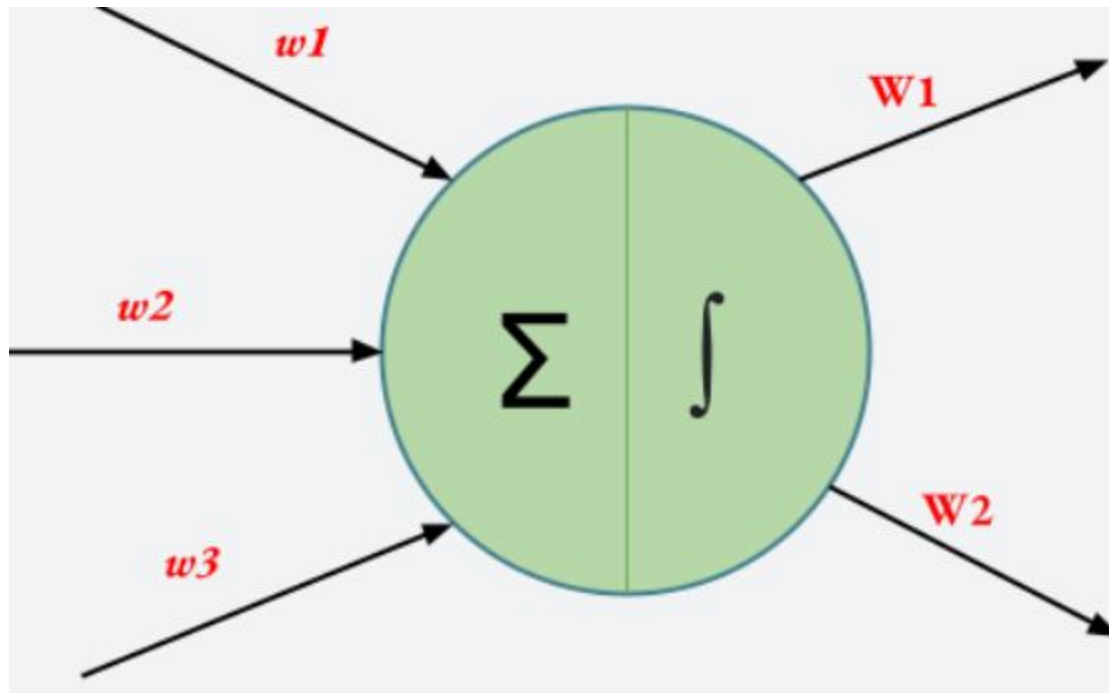
# How A Multi-Layer Neural Network Works

---

- The **inputs** to the network correspond to the attributes measured for each training tuple
- Inputs are fed simultaneously into the units making up the **input layer**
- They are then weighted and fed simultaneously to a **hidden layer**
- The number of hidden layers is arbitrary, although usually only one
- The weighted outputs of the last hidden layer are input to units making up the **output layer**, which emits the network's prediction

# How A Multi-Layer Neural Network Works

- The network is **feed-forward**: None of the weights cycles back to an input unit or to an output unit of a previous layer



# Weight

---

- **1. Zero Initialization**

- As the name suggests, all the weights are assigned zero as the initial value is zero initialization. This kind of initialization is highly ineffective as neurons learn the same feature during each iteration. Rather, during any kind of constant initialization, the same issue happens to occur. Thus, constant initializations are not preferred.

- **2. Random Initialization**

- In an attempt to overcome the shortcomings of Zero or Constant Initialization, random initialization assigns random values except for zeros as weights to neuron paths. However, assigning values randomly to the weights, problems such as Overfitting, Vanishing Gradient Problem, Exploding Gradient Problem might occur.

# Bias

---

- While weights determine the **strength of connections between neurons**, biases provide a critical **additional layer of flexibility** to neural networks. Biases are essentially constants associated with each neuron. Unlike weights, biases are not connected to specific inputs but are added to the neuron's output.
- Biases serve as a form of offset or threshold, allowing neurons to activate even when the weighted sum of their inputs is not sufficient on its own. They introduce a level of adaptability that ensures the network can learn and make predictions effectively.
- To understand the role of biases, consider a simple example. Imagine a neuron that processes the brightness of an image pixel. Without a bias, this neuron might only activate when the pixel's brightness is exactly at a certain threshold. However, by introducing a bias, you allow the neuron to activate even when the brightness is slightly below or above the threshold.
- This flexibility is crucial because real-world data is rarely perfectly aligned with specific thresholds. Biases enable neurons to activate in response to various input conditions, making neural networks more robust and capable of handling complex patterns.
- During training, biases are also adjusted to optimize the network's performance. They can be thought of as fine-tuning parameters that help the network fit the data better.



# Forward Pass

---

- Forward propagation is the initial phase of processing input data through the neural network to produce an output or prediction. Here's how it works:
  1. Input Layer: The input data is fed into the neural network's input layer.
  2. Weighted Sum: Each neuron in the subsequent layers calculates a weighted sum of the inputs it receives, where the weights are the adjustable parameters.
  3. Adding Biases: To this weighted sum, the bias associated with each neuron is added. This introduces an offset or threshold for activation.
  4. Activation Function: The result of the weighted sum plus bias is passed through an activation function. This function determines whether the neuron should activate or remain dormant based on the calculated value.
  5. Propagation: The output of one layer becomes the input for the next layer, and the process repeats until the final layer produces the network's prediction.

# Backpropagation

---

- Iteratively process a set of training tuples & compare the network's prediction with the actual known target value
- For each training tuple, the weights are modified to **minimize the mean squared error** between the network's prediction and the actual target value
- Modifications are made in the “**backwards**” direction: from the output layer, through each hidden layer down to the first hidden layer, hence “**backpropagation**”

# Backpropagation

---

- Steps
  - Initialize weights to small random numbers, associated with biases
  - Propagate the inputs forward (by applying activation function)
  - Backpropagate the error (by updating weights and biases)
  - Terminating condition (when error is very small, etc.)

# Input of Unit J

---

$$I_j = \sum_i w_{ij} O_i + \theta_j,$$

- $w_{ij}$  is the weight of the connection from unit  $i$  in the previous layer to unit  $j$
- $O_i$  is the output of unit  $i$  from the previous layer
- $\theta_j$  is the **bias** of the unit

# Output of Unit J

---

$$O_j = \frac{1}{1 + e^{-I_j}}.$$

# Back propagation of Error

---

For the **Output** layer:

$$Err_j = O_j(1 - O_j)(T_j - O_j),$$

For the **Hidden** layers:

$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk},$$

# Update Weights and Bias

---

Update Weights:

$$\Delta w_{ij} = (l)Err_j O_i.$$

$$w_{ij} = w_{ij} + \Delta w_{ij}.$$

Update Bias:

$$\Delta \theta_j = (l)Err_j.$$

$$\theta_j = \theta_j + \Delta \theta_j.$$

# Example

---

Initial Input, Weight, and Bias Values

| $x_1$ | $x_2$ | $x_3$ | $w_{14}$ | $w_{15}$ | $w_{24}$ | $w_{25}$ | $w_{34}$ | $w_{35}$ | $w_{46}$ | $w_{56}$ | $\theta_4$ | $\theta_5$ | $\theta_6$ |
|-------|-------|-------|----------|----------|----------|----------|----------|----------|----------|----------|------------|------------|------------|
| 1     | 0     | 1     | 0.2      | -0.3     | 0.4      | 0.1      | -0.5     | 0.2      | -0.3     | -0.2     | -0.4       | 0.2        | 0.1        |

$$I_j = \sum_i w_{ij} O_i + \theta_j,$$



# Example

---

## Net Input and Output Calculations

| <i>Unit, <math>j</math></i> | <i>Net Input, <math>I_j</math></i>            | <i>Output, <math>O_j</math></i> |
|-----------------------------|-----------------------------------------------|---------------------------------|
| 4                           | $0.2 + 0 - 0.5 - 0.4 = -0.7$                  | $1/(1 + e^{0.7}) = 0.332$       |
| 5                           | $-0.3 + 0 + 0.2 + 0.2 = 0.1$                  | $1/(1 + e^{-0.1}) = 0.525$      |
| 6                           | $(-0.3)(0.332) - (0.2)(0.525) + 0.1 = -0.105$ | $1/(1 + e^{0.105}) = 0.474$     |

# Example

---

Calculation of the Error at Each Node

| <i>Unit, j</i> | <i>Err<sub>j</sub></i>                       |
|----------------|----------------------------------------------|
| 6              | $(0.474)(1 - 0.474)(1 - 0.474) = 0.1311$     |
| 5              | $(0.525)(1 - 0.525)(0.1311)(-0.2) = -0.0065$ |
| 4              | $(0.332)(1 - 0.332)(0.1311)(-0.3) = -0.0087$ |

# Example

## Calculations for Weight and Bias Updating

| <i>Weight<br/>or Bias</i> | <i>New Value</i>                       |
|---------------------------|----------------------------------------|
| $w_{46}$                  | $-0.3 + (0.9)(0.1311)(0.332) = -0.261$ |
| $w_{56}$                  | $-0.2 + (0.9)(0.1311)(0.525) = -0.138$ |
| $w_{14}$                  | $0.2 + (0.9)(-0.0087)(1) = 0.192$      |
| $w_{15}$                  | $-0.3 + (0.9)(-0.0065)(1) = -0.306$    |
| $w_{24}$                  | $0.4 + (0.9)(-0.0087)(0) = 0.4$        |
| $w_{25}$                  | $0.1 + (0.9)(-0.0065)(0) = 0.1$        |
| $w_{34}$                  | $-0.5 + (0.9)(-0.0087)(1) = -0.508$    |
| $w_{35}$                  | $0.2 + (0.9)(-0.0065)(1) = 0.194$      |
| $\theta_6$                | $0.1 + (0.9)(0.1311) = 0.218$          |
| $\theta_5$                | $0.2 + (0.9)(-0.0065) = 0.194$         |
| $\theta_4$                | $-0.4 + (0.9)(-0.0087) = -0.408$       |

# Neural Network as a Classifier

---

- Weakness
  - Long training time
  - Poor interpretability: Difficult to interpret the symbolic meaning behind the learned weights and of “hidden units” in the network
- Strength
  - High tolerance to noisy data
  - Ability to classify untrained patterns
  - Well-suited for continuous-valued inputs *and outputs*
  - Successful on an array of real-world data, e.g., hand-written letters
  - Algorithms are inherently parallel
  - Techniques have recently been developed for the extraction of rules from trained neural networks

# Class Task

---

- **Input Layer ( $x_1, x_2, x_3, x_4$ ):**
  - Four normalized input neurons represent features of the data.
  - Example values:  $x_1 = 0.7, x_2 = 0.4, x_3 = 1.0, x_4 = 0.6$ .
- **Hidden Layer ( $H_5$  and  $H_6$ ):**
  - Two neurons in the hidden layer:
    - $H_5$ : Receives inputs from all four input neurons ( $w_{15}, w_{25}, w_{35}, w_{45}$ ).
    - $H_6$ : Receives inputs from all four input neurons ( $w_{16}, w_{26}, w_{36}, w_{46}$ ).
  - Biases for  $H_5$  and  $H_6$ :  $\theta_5 = -0.5, \theta_6 = 0.2$ .
- **Output Layer ( $H_7$ ):**
  - A single output neuron ( $H_7$ ) receives inputs from  $H_5$  and  $H_6$  via weights  $w_{57}$  and  $w_{67}$ .
  - Bias for  $H_7$ :  $\theta_7 = 0.3$ .
- **Learning Rate ( $\eta = 0.05$ ):**

